

Player Prefs Enhanced

Documentation | V1.0.0 | 23-03-22



Table of Contents

1. Get started quickly	3
2. Introduction	4
3. Changelog	5
4. Set-Up	6
5. Editor	7
6. API	9
7. Known Limitations	10
8. Support and feedback	11

1. Get started quickly

To use your player prefs in code, import the **DTT.PlayerprefsEnhanced** namespace, now you can use **EnhancedPrefs** instead of Unity's **PlayerPrefs**. The player prefs you create will be encrypted by default, to make them more secure you should change the default key and IV values in the **EncryptionAES** script.

To make use of the **Player Pref Overview**, the 'API Compatibility Level' needs to be set to ".Net 4.x". This setting can be found in **Edit/Project Settings/Player**.

To open the player pref overview, go to the menu item **Tools/Player Prefs Enhanced/Player Prefs Overview**. In this window, you can freely add, remove and edit your player prefs.

2. Introduction

DTT Player Prefs Enhanced is an asset for the Unity Engine that allows you to manage your player prefs. The asset is meant to replace the use of Unity's **PlayerPref** class in code with DTT's **EnhancedPrefs** class.

DTT **Player Prefs Enhanced** allows you to store any serializable type where Unity would only allow you to store **int**, **float** and **string** values. Another difference is the possibility to encrypt the stored player prefs.

To make the managing of player prefs even easier, the asset provides the **Player Prefs Overview** window. In this window, you can create new player prefs and edit or delete existing ones. You can also search for a specific player pref with the search and sort options.

3. Changelog

1. version 1.0.0 – Initial release

4. Set-Up

To use your player prefs in code, import the **DTT.PlayerprefsEnhanced** namespace, now you can use **EnhancedPrefs** instead of Unity's **PlayerPrefs**. The class provides a similar API, where instead of **SetString()** etc., you can use **SetPlayerPref()** to store your player prefs. By default, the player prefs will be encrypted to avoid this you need to use the overloaded method to enter a boolean value that determines the encryption.

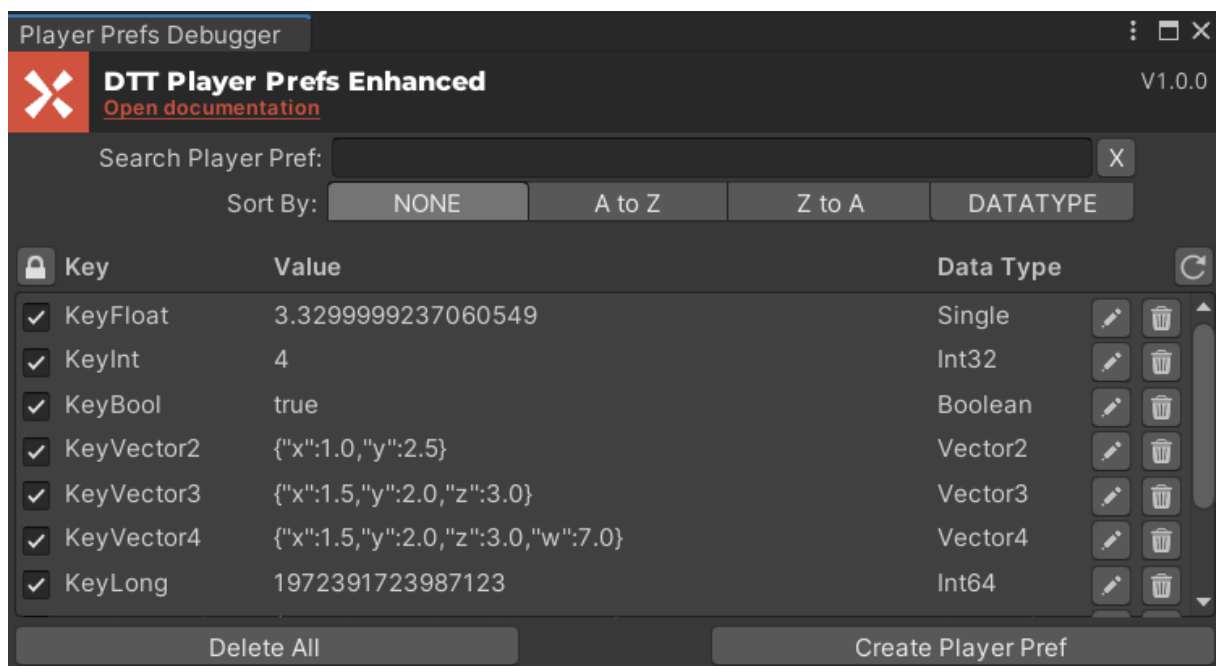
To make them more secure you should change the default key and IV values in the **EncryptionAES** script. The key must have a value of 32 characters long and the IV a value of 16 characters long.

To make the managing of player prefs even easier, the asset provides the **Player Prefs Overview** window. If you want to make use of the player pref overview, the '**API Compatibility Level**' needs to be set to ".Net 4.x". This setting can be found in **Edit/Project Settings/Player**.

To open the player pref overview, go to the menu item **Tools/Player Prefs Enhanced/Player Prefs Overview**. In this window, you can freely add, remove and edit your player prefs.

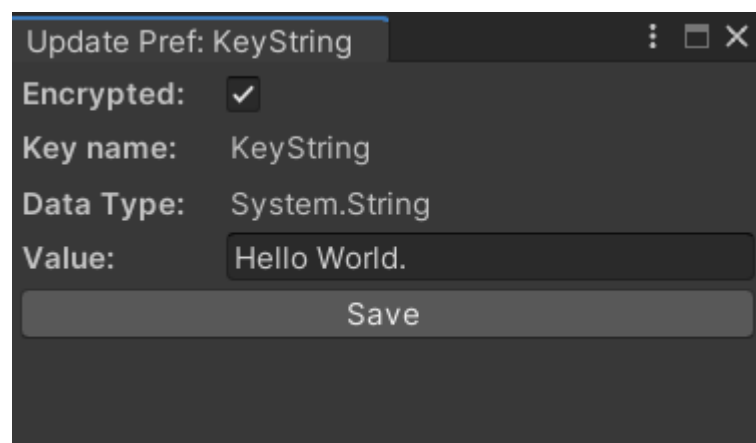
5. Editor

The **Player Pref Overview window** is used to display all currently stored player prefs. Within this window, you can easily search for your player pref by key name or datatype.

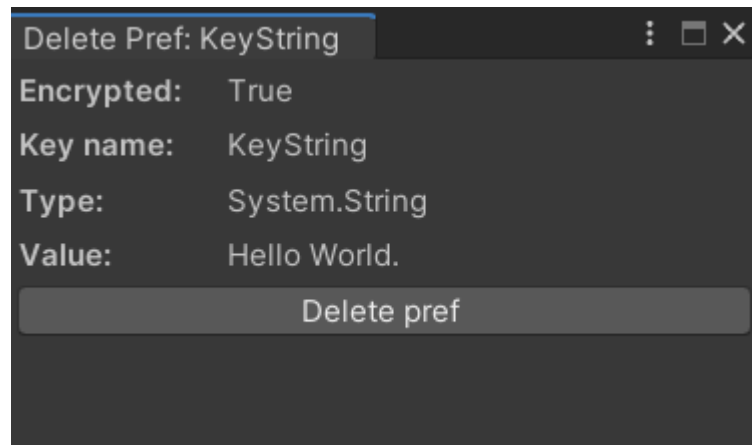


For every player pref the key, value and data type will be displayed. Each player pref has two buttons, one to edit the player pref value and one to delete the player pref. Using one of these two buttons will open a new window where the player pref info will be displayed.

In the edit window, the value of the pref can be changed and saved.

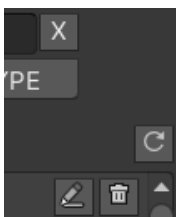
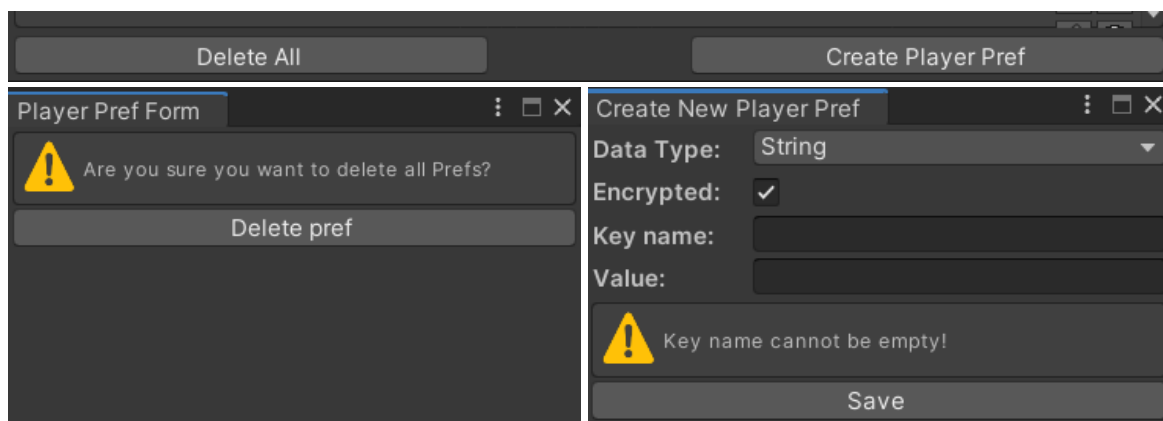


In the delete window, the player pref value can only be deleted.



At the bottom of the window, two buttons can be found. The 'Delete all' button opens a window asking for confirmation. After you confirm that you want to delete all player prefs, all stored player prefs will be deleted.

The 'Create Player Pref' button opens a window in which you can create a new player pref and save it. In this window, you can select the desired data type of your pref and fill in the input fields. If no warnings are displayed the pref can be saved, the window will close and the list of player prefs will be updated. Otherwise, no changes will be made.



If player prefs are altered within code while the **Player Pref Overview** was open, you might need to refresh the list to have the changes displayed.

6. API

EnhancedPrefs

```
// Key used to store and get the player pref value.
private string _myPrefKey = "Key";

// Value to be held by the player pref, can be any serializable class.
private string _myPrefValue = "Value";

// Value to be held by the player pref, can be any serializable class.
private string _myDefaultValue = "Default";

// True if player pref should be encrypted.
private bool _encryptPref = false;

private void Start()
{
    // Encrypts and stores the value of a player pref.
    // If the key does not exist a new player pref will be stored.
    EnhancedPrefs.SetPlayerPref(_myPrefKey, _myPrefValue);

    // Stores the value of a player pref.
    // If the key does not exist a new player pref will be stored.
    EnhancedPrefs.SetPlayerPref(_myPrefKey, _myPrefValue, _encryptPref);

    // Returns the stored player pref value.
    // If the key does not exist the default value is returned.
    string playerPrefValue = EnhancedPrefs.GetPlayerPref(_myPrefKey, _myDefaultValue);

    // Whether a player pref with this key is stored or not.
    bool hasKey = EnhancedPrefs.HasPlayerPref(_myPrefKey);

    // Write all modified player prefs to disk.
    EnhancedPrefs.SavePlayerPrefs();

    // Removes the player pref with the same key as provided.
    EnhancedPrefs.DeletePlayerPref(_myPrefKey);

    // Deletes all stored player prefs.
    EnhancedPrefs.DeleteAllPlayerPrefs();
}
```

7. Known Limitations

- The Player Pref Overview can only be used when the 'API Compatibility Level' is set to ".Net 4.x".

8. Support and feedback

If you have any questions regarding the use of this asset, we are happy to help you out.

Always feel free to contact us at:

unity-support@d-tt.nl

(We typically respond within 1-2 business days)

We are actively developing this asset, with many future updates and extensions already planned. We are eager to include feedback from our users in future updates, be they 'quality of life' improvements, new features, bug fixes or anything else that can help you improve your experience with this asset. You can reach us at the email above.

Reviews and ratings are very much appreciated as they help us raise awareness and to improve our assets.

DTT stands for Doing Things Together

DTT is an app, web and game development agency based in the centre of Amsterdam. Established in 2010, DTT has over a decade of experience in mobile, game, and web based technology.

Our game department primarily works in Unity where we put significant emphasis on the development of internal packages, allowing us to efficiently reuse code between projects. To support the Unity community, we are publishing a selection of our internal packages on the Asset Store, including this one.

More information about DTT (including our clients, projects and vacancies) can be found here:

<https://www.d-tt.nl/en/>